

**LAPORAN SIMULASI TIME SYNCHRONIZATION
MENGUNAKAN LAMPORT CLOCK DAN VECTOR
CLOCK**



**DIPERSIAPKAN OLEH:
SISTEM TERDISTRIBUSI**

**BRADLEY WIDJAJA
5025231036**

Teknik Informatika
Fakultas Teknik Elektro dan Informatika Cerdas
Institut Teknologi Sepuluh Nopember
2025

Pendahuluan

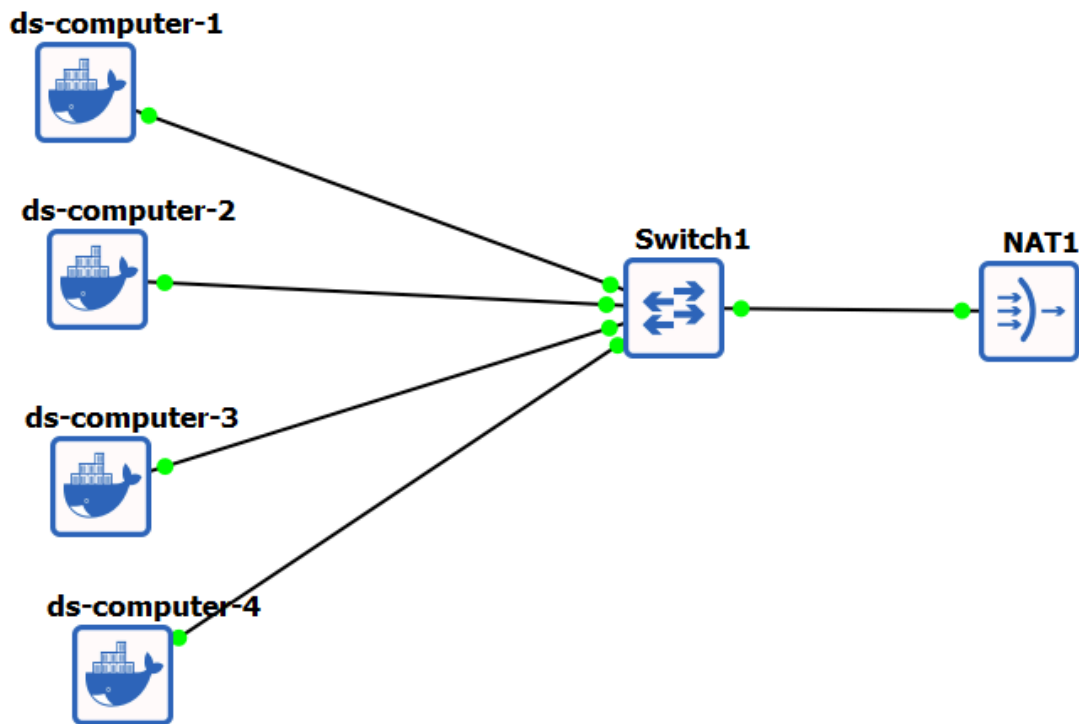
Pada suatu sistem terdistribusi, setiap node memiliki local clock yang dapat memiliki kemungkinan untuk berbeda dikarenakan clock drift. Perbedaan waktu ini akhirnya menyebabkan terjadinya error atau kesalahan dalam proses logging, ordering events, message passing, dan proses yang lainnya. Oleh karena itu, untuk mengatasi masalah tersebut, dibutuhkan suatu logical clock meliputi Lamport Clock dan Vector Clock yang mampu menjaga konsistensi hubungan causal ordering antar node tanpa harus melakukan sinkronisasi waktu fisik.

Dalam tugas ini, dilakukan simulasi time synchronization menggunakan Lamport Clock dan Vector Clock menggunakan GNS3 dengan komponen sistem berjalan secara terpisah di beberapa node dan berkomunikasi menggunakan protokol tertentu. Eksperimen dijalankan dengan 4 node berupa node_a, node_b, node_c, dan node_d dengan node_c yang memiliki peran sebagai logger.

Tugas yang Diberikan

1. Menjalankan simulasi time synchronization menggunakan Lamport Clock dan Vector Clock dari repository git:
https://github.com/rm77/ds25/tree/main/synchronization/time_sync
2. Menjalankan komponen logger, node_a, node_b, node_c, dan node_d menggunakan GNS3.
3. Mendokumentasikan cara menjalankan eksperimen disertai dengan screenshot.
4. Menjelaskan arsitektur percobaan, protokol komunikasi, dan mekanisme time synchronization.

Topologi



Penjelasan:

Topologi terdiri dari 4 hostname yang berbasis docker container, yaitu ds-computer-1 sebagai node_a, ds-computer-2 sebagai node_b, ds-computer-3 sebagai node_c yang berperan sebagai logger, dan ds-computer-4 sebagai node_d. Keempat node tersebut terhubung ke Switch1. Konfigurasi ini dijalankan pada broadcast domain yang sama yang menandakan node dapat saling berkomunikasi tanpa harus melalui perangkat routing. Seperti yang telah disinggung sebelumnya, node logger terletak pada node_c yang berfungsi untuk menerima, menyimpan, dan melakukan analisis message dari node_a, node_b, dan node_d, termasuk timestamp Lamport dan Vector Clock. Dengan log ini, dapat dilakukan evaluasi terhadap hubungan causal order antar event, mendeteksi concurrent event, dan mengidentifikasi dampak dari time drift pada alur komunikasi terdistribusi.

Spesifikasi

Hostname	Node	IP	Port	Spesifikasi
ds-computer-1	Node A	192.168.122.10	5000	x86_64 • Intel® Core™ i5-1235U • 1 Core / 1 Thread • 2 GiB RAM • 20–30 GB Storage • Debian (GNS3 VM) • KVM/VMware
ds-computer-2	Node B	192.168.122.11	5001	x86_64 • Intel® Core™ i5-1235U • 1 Core / 1 Thread • 2 GiB RAM • 20–30 GB Storage • Debian (GNS3

				VM) • KVM/VMware
ds-computer-3	Node C (Logger)	192.168.122.12	5002	x86_64 • Intel® Core™ i5-1235U • 1 Core / 1 Thread • 2 GiB RAM • 20–30 GB Storage • Debian (GNS3 VM) • KVM/VMware
ds-computer-4	Node D	192.168.122.13	5003	x86_64 • Intel® Core™ i5-1235U • 1 Core / 1 Thread • 2 GiB RAM • 20–30 GB Storage • Debian (GNS3 VM) • KVM/VMware

Repository

<https://github.com/bwbrad05/Tugas-1-Sistem-Terdistribusi.git>

Penjelasan Setiap Penggunaan

1. Setup Topologi pada GNS 3
 - Membuat topologi jaringan pada GNS3 menggunakan 4 node berbasis docker container yang terhubung ke 1 switch.
 - Menambahkan perangkat NAT yang terhubung ke switch untuk menyediakan koneksi internet agar dapat melakukan cloning dari repository git.
2. Melakukan configure IP statis pada node masing - masing. IP dikonfigurasi dalam subnet NAT lengkap dengan gateway (192.168.122.1) dan DNS Nameserver (8.8.8.8) yang berfungsi untuk menghubungkan dengan koneksi internet.
3. Melakukan git clone dari repository yang telah diberikan pada setiap node dengan command git clone (untuk tugas ini dapat menggunakan <https://github.com/bwbrad05/Tugas-1-Sistem-Terdistribusi.git>)
4. Pada pc masing - masing, diarahkan menuju ke directory masing - masing node sesuai dengan pc yang digunakan (misal ds-computer-1 menggunakan directory node_a dan seterusnya).
5. Menjalankan logger pada Node C dengan command ./run.bash

Code pada run.bash:

```
#!/bin/bash
```

```
python3 logger.py --bind 0.0.0.0 --port 9999 --expect 10
```

- --bind 0.0.0.0 berfungsi agar logger dapat mendengarkan di semua network interface.
- --port 9999 menandakan setting port 9999 untuk menerima log event dari node - node yang lain
- --expect 10 menandakan setting untuk menunggu 10 chat messages sebelum mencetak ordering.

6. Jalankan node - node lain yang akan melakukan komunikasi melalui run.bash (node_a, node_b, dan node_c) dengan command yang sama yaitu ./run.bash

a. Node A

Code pada run.bash:

```
#!/bin/bash
```

```
python3 peer_node.py \  
    --name A \  
    --listen 192.168.122.10 5000 \  
    --peers A@192.168.122.10:5000 B@192.168.122.11:5001  
D@192.168.122.13:5002 \  
    --logger 192.168.122.12 9999 \  
    --offset-ms 600 \  
    --initiate-broadcast --msg "Hello from A"
```

- --name A berfungsi untuk memberikan identitas kepada node A.
- --listen 192.168.122.10 5000 berfungsi untuk listen di port 5000
- --peers digunakan untuk memberikan list semua peer nodes termasuk node A sendiri.
- --logger 192.168.122.12 9999 berfungsi untuk mengirim log kepada logger yang terletak pada IP dan port yang ditentukan.
- --offset-ms 600 menandakan time drift +600 ms yang berarti clock lebih cepat 600 ms dari default.
- --initiate-broadcast --msg "Hello from A" berfungsi untuk melakukan broadcast pada seluruh node dengan message yang disampaikan berupa "Hello from A".

b. Node B

Code:

```
#!/bin/bash
```

```
python3 peer_node.py \  
    --name B \  
    --listen 192.168.122.11 5001 \  
    --peers A@192.168.122.10:5000 B@192.168.122.11:5001  
D@192.168.122.13:5002 \  
    --logger 192.168.122.12 9999 \  
    --offset-ms 300
```

- Konfigurasi parameter pada node B hampir sama dengan A, hanya berbeda pada offset.
- --name B berfungsi untuk memberikan identitas kepada node B.
- --listen 192.168.122.11 5001 berfungsi untuk listen di port 5001

- --offset-ms 300 menandakan time drift +300 ms yang berarti clock lebih cepat 300 ms lebih cepat dari default.

c. Node D

Code:

```
#!/bin/bash
```

```
python3 peer_node.py \
    --name D \
    --listen 192.168.122.13 5002 \
    --peers A@192.168.122.10:5000 B@192.168.122.11:5001
D@192.168.122.13:5002 \
    --logger 192.168.122.12 9999 \
    --offset-ms -500
```

- Konfigurasi parameter pada node D hampir sama dengan A dan B, hanya berbeda pada offset.
- --name D berfungsi untuk memberikan identitas kepada node D.
- --listen 192.168.122.13 5002 berfungsi untuk listen di port 5002
- --offset-ms - 500 menandakan time drift -500 ms yang berarti clock lebih cepat 500 ms lebih lambat dari default.

Penjelasan Komunikasi Antar Node

Protokol Transport

Dilihat dari kode yang ada dalam peer_node.py, sistem berkomunikasi menggunakan protokol transport UDP (User Datagram Protocol). UDP dipilih karena karakteristiknya yang ringan, yaitu tidak perlu adanya proses handshake sehingga pengiriman pesan dapat dilakukan secara langsung dan lebih cepat dan pesan dikirim apa adanya karena simulasi hanya untuk memperlihatkan causal ordering yang lebih berfokus pada urutan event dan tidak kepada reliabilitas. Selain itu, latency yang dimiliki juga rendah sehingga event yang terjadi antar node dapat dicatat dengan lebih cepat dan analisis Lamport dan Vector clock dapat dilakukan dengan efektif.

Protokol Message Exchange

Format Pesan

Semua pesan yang akan dikirim dan diterima dibungkus dalam bentuk JSON yang berisikan:

- Informasi pengirim dan penerima
- Jenis pesan berupa chat atau ACK.
- Isi pesan teks dengan mekanisme berupa:
 - a. Unicast: Jika ingin mengirimkan pesan dari satu node kepada satu node spesifik (format: @NAME message)

- b. Broadcast: Jika ingin mengirimkan pesan dari satu node menuju seluruh node lainnya sekaligus (format: /broadcast message).
- Logical clock berupa lamport dan vector clock:
 - a. Lamport Clock (L): Clock logika yang akan dilakukan increment setiap terjadinya suatu event.
 - b. Vector Clock (V): Sebuah dictionary yang menyimpan clock masing - masing node.
- Timestamp lokal dan monotonik.

Dengan format tersebut, dapat dipastikan bahwa setiap pesan membawa informasi causal ordering yang lengkap.

Jenis Pesan

Jika jenis pesan yang dikirimkan merupakan chat, maka menandakan hal tersebut merupakan pesan yang akan disampaikan antar node. Di lain sisi, jika pesan berupa ACK, hal tersebut merupakan pesan konfirmasi bahwa chat telah berhasil diterima. Adanya ACK dikarenakan protokol UDP tidak terlalu reliable.

Protokol Time Synchronization

Sistem menggunakan logical time synchronization. Berikut merupakan alur pengiriman dan penerimaan pesan dengan menggunakan protokol logical time synchronization:

a. Pengiriman Pesan

Saat node mengirimkan pesan:

- Lamport clock akan diincrement.
- Vector clock untuk node tersebut akan diincrement.
- Node membuat payload JSON berisi semua informasi clock
- Seluruh komunikasi antar node akan secara otomatis dikirimkan juga menuju ke logger yang sudah diaktifkan.

b. Penerimaan Pesan

Saat node menerima pesan:

- Node membaca payload JSON.
- Node memperbarui Lamport Clock dengan aturan:

$$L = \max(L_local, L_incoming) + 1$$
- Node memperbarui Vector clock dengan mengambil nilai maksimum tiap entri, lalu increment komponen sendiri:

$$V_local[i] = \max(V_local[i], V_incoming[i])$$

$$V_local[self] += 1$$
- Node mencatat log penerimaan beserta perubahan clock.

Penjelasan Simulasi Time Drift

Setiap node diberikan waktu local yang berbeda menggunakan offset waktu buatan:

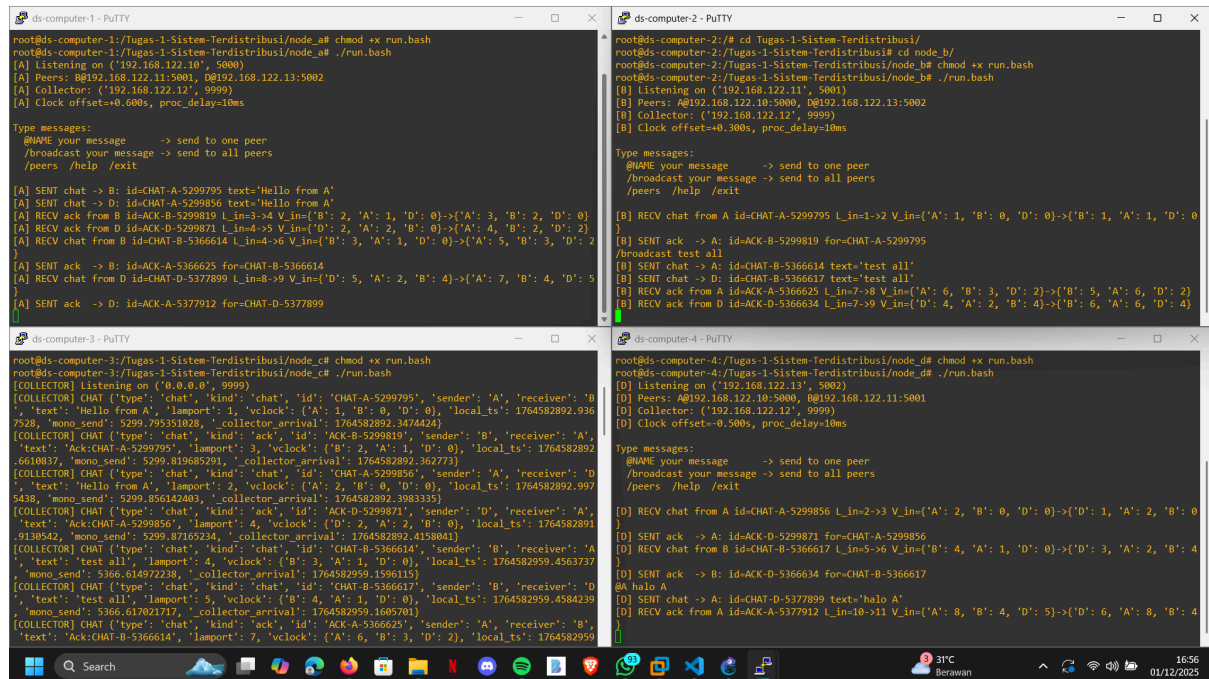
--offset-ms <value>

Offset ini ditambahkan ke `time.time()` untuk menghasilkan timestamp lokal:

`local_ts = time.time() + offset_s`

Untuk simulasi yang dilakukan, dilakukan setting untuk offset dari masing - masing node berupa Node A memiliki +600 ms (600 ms lebih cepat), Node B +300 ms (300 ms lebih cepat), dan Node C memiliki -500 ms (500 ms lebih lambat).

Screenshot Simulasi



```
root@ds-computer-1:~/Tugas-1-Sistem-Terdistribusi/node_a# chmod +x run.bash
root@ds-computer-1:~/Tugas-1-Sistem-Terdistribusi/node_a# ./run.bash
[A] Listening on ('192.168.122.10', 5000)
[A] Peers: B@192.168.122.11:5001, D@192.168.122.13:5002
[A] Collector: ('192.168.122.12', 9999)
[A] Clock offset=+6.600s, proc_delay=10ms

Type messages:
@NAME your message -> send to one peer
/broadcast your message -> send to all peers
/peers /help /exit

[A] SENT chat -> B: id=CHAT-A-5299795 text='Hello from A'
[A] SENT chat -> D: id=CHAT-A-5299856 text='Hello from A'
[A] RECV ack from B id=ACK-B-5299819 L_in=3->4 V_in={'B': 2, 'A': 1, 'D': 0}->{'A': 3, 'B': 2, 'D': 0}
[A] RECV ack from D id=ACK-D-5299871 L_in=4->5 V_in={'D': 2, 'A': 2, 'B': 0}->{'A': 4, 'B': 2, 'D': 2}
[A] RECV chat from B id=CHAT-B-5366614 L_in=4->6 V_in={'B': 3, 'A': 1, 'D': 0}->{'A': 5, 'B': 3, 'D': 2}
[A] SENT ack -> D: id=ACK-A-5366625 for=CHAT-B-5366614
[A] RECV chat from D id=CHAT-D-5377899 L_in=8->9 V_in={'D': 5, 'A': 2, 'B': 4}->{'A': 7, 'B': 4, 'D': 5}
[A] SENT ack -> D: id=ACK-A-5377912 for=CHAT-D-5377899

root@ds-computer-2:~/Tugas-1-Sistem-Terdistribusi/# cd Tugas-1-Sistem-Terdistribusi/
root@ds-computer-2:~/Tugas-1-Sistem-Terdistribusi# cd node_b/
root@ds-computer-2:~/Tugas-1-Sistem-Terdistribusi/node_b# chmod +x run.bash
root@ds-computer-2:~/Tugas-1-Sistem-Terdistribusi/node_b# ./run.bash
[B] Listening on ('192.168.122.11', 5001)
[B] Peers: A@192.168.122.10:5000, D@192.168.122.13:5002
[B] Collector: ('192.168.122.12', 9999)
[B] Clock offset=+0.300s, proc_delay=10ms

Type messages:
@NAME your message -> send to one peer
/broadcast your message -> send to all peers
/peers /help /exit

[B] RECV chat from A id=CHAT-A-5299795 L_in=1->2 V_in={'A': 1, 'B': 0, 'D': 0}->{'B': 1, 'A': 1, 'D': 0}
[B] SENT ack -> A: id=ACK-B-5299819 for=CHAT-A-5299795
/broadcast test all
[B] SENT chat -> A: id=CHAT-B-5366614 text='test all'
[B] SENT chat -> D: id=CHAT-B-5366617 text='test all'
[B] RECV ack from A id=ACK-A-5366625 L_in=7->8 V_in={'A': 6, 'B': 3, 'D': 2}->{'B': 5, 'A': 6, 'D': 2}
[B] RECV ack from D id=ACK-D-5366634 L_in=7->9 V_in={'D': 4, 'A': 2, 'B': 4}->{'B': 6, 'A': 6, 'D': 4}

root@ds-computer-3:~/Tugas-1-Sistem-Terdistribusi/node_c# chmod +x run.bash
root@ds-computer-3:~/Tugas-1-Sistem-Terdistribusi/node_c# ./run.bash
[COLLECTOR] Listening on ('0.0.0.0', 9999)
[COLLECTOR] CHAT ('type': 'chat', 'kind': 'chat', 'id': 'CHAT-A-5299795', 'sender': 'A', 'receiver': 'B', 'text': 'Hello from A', 'lampoort': 1, 'vclock': {'A': 1, 'B': 0, 'D': 0}, 'local_ts': 1764582892.9367528, 'mono_send': 5299.795351028, 'collector_arrival': 1764582892.3474424)
[COLLECTOR] CHAT ('type': 'chat', 'kind': 'ack', 'id': 'ACK-B-5299819', 'sender': 'B', 'receiver': 'A', 'text': 'Ack:CHAT-A-5299795', 'lampoort': 2, 'vclock': {'D': 2, 'A': 1, 'D': 0}, 'local_ts': 1764582892.6610837, 'mono_send': 5299.819685291, 'collector_arrival': 1764582892.362773)
[COLLECTOR] CHAT ('type': 'chat', 'kind': 'chat', 'id': 'CHAT-A-5299856', 'sender': 'A', 'receiver': 'D', 'text': 'Hello from A', 'lampoort': 2, 'vclock': {'A': 2, 'B': 0, 'D': 0}, 'local_ts': 1764582892.9979438, 'mono_send': 5299.856142403, 'collector_arrival': 1764582892.398333)
[COLLECTOR] CHAT ('type': 'chat', 'kind': 'ack', 'id': 'ACK-D-5299871', 'sender': 'D', 'receiver': 'A', 'text': 'Ack:CHAT-A-5299856', 'lampoort': 4, 'vclock': {'D': 2, 'A': 2, 'B': 0}, 'local_ts': 1764582891.9130542, 'mono_send': 5299.87165234, 'collector_arrival': 1764582892.4158041)
[COLLECTOR] CHAT ('type': 'chat', 'kind': 'chat', 'id': 'CHAT-B-5366614', 'sender': 'B', 'receiver': 'A', 'text': 'test all', 'lampoort': 4, 'vclock': {'B': 3, 'A': 1, 'D': 0}, 'local_ts': 1764582959.4563737, 'mono_send': 5366.614972238, 'collector_arrival': 1764582959.1596115)
[COLLECTOR] CHAT ('type': 'chat', 'kind': 'chat', 'id': 'CHAT-B-5366617', 'sender': 'B', 'receiver': 'D', 'text': 'test all', 'lampoort': 5, 'vclock': {'B': 4, 'A': 1, 'D': 0}, 'local_ts': 1764582959.4584239, 'mono_send': 5366.617021717, 'collector_arrival': 1764582959.1605701)
[COLLECTOR] CHAT ('type': 'chat', 'kind': 'ack', 'id': 'ACK-A-5366625', 'sender': 'A', 'receiver': 'B', 'text': 'Ack:CHAT-B-5366614', 'lampoort': 7, 'vclock': {'A': 6, 'B': 3, 'D': 2}, 'local_ts': 1764582959.1605701)

root@ds-computer-4:~/Tugas-1-Sistem-Terdistribusi/node_d# chmod +x run.bash
root@ds-computer-4:~/Tugas-1-Sistem-Terdistribusi/node_d# ./run.bash
[D] Listening on ('192.168.122.13', 5002)
[D] Peers: A@192.168.122.10:5000, B@192.168.122.11:5001
[D] Collector: ('192.168.122.12', 9999)
[D] Clock offset=-0.500s, proc_delay=10ms

Type messages:
@NAME your message -> send to one peer
/broadcast your message -> send to all peers
/peers /help /exit

[D] RECV chat from A id=CHAT-A-5299856 L_in=2->3 V_in={'A': 2, 'B': 0, 'D': 0}->{'D': 1, 'A': 2, 'B': 0}
[D] SENT ack -> A: id=ACK-D-5299871 for=CHAT-A-5299856
[D] RECV chat from B id=CHAT-B-5366617 L_in=5->6 V_in={'B': 4, 'A': 1, 'D': 0}->{'D': 3, 'A': 2, 'B': 4}
[D] SENT ack -> B: id=ACK-D-5366634 for=CHAT-B-5366617
@A halo A
[D] SENT chat -> A: id=CHAT-D-5377899 text='halo A'
[D] RECV ack from A id=ACK-A-5377912 L_in=10->11 V_in={'A': 8, 'B': 4, 'D': 5}->{'D': 6, 'A': 8, 'B': 4}
```

Penjelasan:

Screenshot berikut menunjukkan aktivitas terjadinya proses komunikasi antar node yang berjalan secara bersamaan dengan setiap pengiriman dan penerimaan pesan dicatat dengan Lamport dan Vector Clock. Saat sebuah node mengirimkan pesan, Lamport clock-nya akan diincrement dan juga ketika menerima pesan, nilai clocknya diperbarui menjadi $\max(\text{local}, \text{received}) + 1$, sesuai dengan penjelasan pada bagian protokol time synchronization. Selain itu, Vector clock juga ikut diperbarui dengan mengambil nilai maksimum setiap indeks lalu menambah indeks milik node pengirim sehingga terbentuk jejak kausal yang jelas pada log setiap node. Hal tersebut dapat dilihat dari logger ketika awal broadcast A, Vector clock masih $\{A': 2, B': 0, D': 0\}$ yang menandakan A berhasil melakukan broadcast kepada 2 node lainnya dan setelah B dan D berhasil menerima dan send kembali ACK ke A sehingga clock B dan D masing - masing bertambah 2. Vector clock akan berubah menjadi $\{A': 4, B': 2, D': 2\}$ dengan A menjadi 4 karena +2 ACK dari B dan D.

Screenshot Time Drift


```
[COLLECTOR] Listening on ('0.0.0.0', 9999)
[COLLECTOR] CHAT {'type': 'chat', 'kind': 'chat', 'id': 'CHAT-A-5299795', 'sender': 'A', 'receiver': 'A', 'text': 'Hello from A', 'lambport': 1, 'vclock': {'A': 1, 'B': 0, 'D': 0}, 'local_ts': 1764582892.937528, 'mono_send': 5299.795351028, '_collector_arrival': 1764582892.3474424}
[COLLECTOR] CHAT {'type': 'chat', 'kind': 'ack', 'id': 'ACK-B-5299819', 'sender': 'B', 'receiver': 'A', 'text': 'Ack:CHAT-A-5299795', 'lambport': 3, 'vclock': {'B': 2, 'A': 1, 'D': 0}, 'local_ts': 1764582892.6610837, 'mono_send': 5299.819685291, '_collector_arrival': 1764582892.362773}
[COLLECTOR] CHAT {'type': 'chat', 'kind': 'chat', 'id': 'CHAT-A-5299856', 'sender': 'A', 'receiver': 'A', 'text': 'Hello from A', 'lambport': 2, 'vclock': {'A': 2, 'B': 0, 'D': 0}, 'local_ts': 1764582892.995438, 'mono_send': 5299.856142403, '_collector_arrival': 1764582892.3983335}
[COLLECTOR] CHAT {'type': 'chat', 'kind': 'ack', 'id': 'ACK-D-5299871', 'sender': 'D', 'receiver': 'A', 'text': 'Ack:CHAT-A-5299856', 'lambport': 4, 'vclock': {'D': 2, 'A': 2, 'B': 0}, 'local_ts': 1764582892.9130542, 'mono_send': 5299.87165234, '_collector_arrival': 1764582892.4158041}
```

Penjelasan:

Pada log yang dicatat logger tersebut dapat terlihat jelas time drift karena `local_ts` dan `mono_send` yang tidak sinkron sama sekali antara node yang satu dengan node yang lain. Dapat dilihat dari hasil screenshot, pesan pertama dari A memiliki `local_ts` = 1764582892.93 dan `mono_send` = 5299.79 dan disusul dengan ACK dari B dengan `local_ts` = 1764582892.66 dan `mono_send` = 5299.81 padahal secara kausal, pesan ACK dari B adalah respons langsung ke A. Perbedaan nilai tersebut menunjukkan bahwa clock fisik B berjalan dengan kecepatan berbeda dari A sehingga timestamp-nya tidak lagiurut secara real-time. Pesan dari D yang kemudian muncul dengan `local_ts` = 1764582893.91 yang semakin menegaskan setiap node memiliki offset yang berbeda. Meskipun begitu, dapat dilihat dari `_collector arrival` dan `mono_send` yang monoton naik, log tersebut masih menunjukkan pengiriman secaraurut meskipun bukan urutan secara kausal. Namun, utamanya, penggunaan Lamport dan Vector clock yang sangat berpengaruh karena mampu memulihkan urutan kausal yang hilang akibat dari terjadinya time drift.

Kesimpulan

Suatu sistem yang tidak menggunakan logical clock akan tidak dapat mengurutkan event sesuai dengan causal order yang benar dikarenakan time drift dari masing - masing node yang berbeda. Logger yang mencatat urutan event akan menjadi kacau, seperti message yang dikirimkan bisa jadi masuk sebelum bahkan message tersebut dikirimkan. Oleh karena itu, dengan adanya logical clock berupa Lamport clock dan Vector clock membuat sistem menjadi konsisten meskipun terdapat perbedaan timestamp fisik antar masing - masing node. Lamport clock bekerja dengan cara menaikkan counter setiap kali terjadi event send atau receive, sehingga semua event memiliki urutan global yang monoton naik, dan Vector clock menyimpan array counter untuk setiap node sehingga sistem dapat mengetahui apakah dua event berhubungan secara kausal atau terjadi secara paralel. Dengan penggunaan logical clock tersebut akhirnya menyediakan timestamp yangurut dan monoton sehingga semua node memiliki pemahaman yang sama terhadap urutan event yang terjadi. Dengan demikian, hubungan sebab-akibat antar pesan tetap terjaga dengan akurat dan sistem dapat mengidentifikasi event berurutan maupun concurrent tanpa dipengaruhi oleh adanya perbedaan waktu dari masing - masing node.